Healthcare Appointment Scheduling System

Sprint 2 Report: Database Implementation & UI-Based System Validation

Team Name	Healthcare Database Team
Team Members	Caiden Sanders, Vsevolod Kovalev
Course	CPT S 451 – Spring 2026
Prepared For	Sprint 2 assignment submission

Report scope

This report consolidates the team's prior design and implementation work into the Sprint 2 deliverable. It covers SQL implementation, business rule enforcement, edge-case handling, normalization, CRUD operations, advanced retrieval queries, validation with sample data, and UI-based system validation notes.

1. Assignment Overview

The goal of Sprint 2 is to implement the Healthcare Appointment Scheduling System in SQL and demonstrate that the system enforces business rules, rejects invalid data, handles edge cases, supports real-world CRUD operations, and can be exercised through a basic user interface.

This document is organized so that the sections map directly to the assignment rubric: database implementation, business rules, edge cases, normalization, SQL operations, advanced queries, validation with sample data, and UI support.

Requirement	Description	Primary Supporting Tables
FR-1	Users create an account	USER
FR-2	Users log in with credentials	USER
FR-3	Administrators add new records	USER, PATIENT, PROVIDER, DEPARTMENT, APPOINTMENT
FR-4	Users view stored data	PATIENT, PROVIDER, APPOINTMENT, DEPARTMENT
FR-5	Users update records	USER, PATIENT, PROVIDER, APPOINTMENT
FR-6	Administrators delete records safely	PATIENT, PROVIDER, APPOINTMENT, DEPARTMENT
FR-7	All system data is stored in a structured database	All tables
FR-8	Patients schedule appointments	APPOINTMENT, PATIENT, PROVIDER
FR-9	Patients cancel or reschedule appointments	APPOINTMENT, APPOINTMENT_AUDIT_LOG
FR-10	Providers view scheduled appointments	APPOINTMENT, PROVIDER
FR-11	System prevents scheduling conflicts	APPOINTMENT, PROVIDER_AVAILABILITY, app transaction logic
FR-12	Administrators generate reports	All tables via JOIN queries

2. Database Implementation

The database uses a relational design centered on appointments. Shared account information is stored in USER, role-specific details are stored in PATIENT and PROVIDER, and supporting lookup and history data are stored in DEPARTMENT, APPOINTMENT_STATUS, PROVIDER_AVAILABILITY, and APPOINTMENT_AUDIT_LOG.

Table	Purpose	Primary Key	Key Foreign Keys	Critical Constraints
USER	Stores shared account and authentication data for all users.	user_id	None	UNIQUE(email), CHECK(role), NOT NULL
PATIENT	Stores patient-specific	patient_id	user_id → USER	UNIQUE(user_id), CHECK(gender)

Table	Purpose	Primary Key	Key Foreign Keys	Critical Constraints
	demographic, emergency, and insurance data.			
DEPARTMENT	Stores clinical departments and contact information.	department_id	None	UNIQUE(department_name)
PROVIDER	Stores provider-specific profile and department assignment.	provider_id	user_id → USER; department_id → DEPARTMENT	UNIQUE(user_id), UNIQUE(license_number)
APPOINTMENT_STATUS	Lookup table for allowed appointment statuses.	status_id	None	UNIQUE(status_name)
APPOINTMENT	Core transaction table linking patient, provider, department, and status.	appointment_id	patient_id, provider_id, department_id, status_id	CHECK(end_time > start_time), defaults
PROVIDER_AVAILABILITY	Stores recurring provider work windows by day of week.	availability_id	provider_id → PROVIDER	CHECK(day_of_week), CHECK(end_time > start_time)
APPOINTMENT_AUDIT_LOG	Tracks appointment status changes for accountability.	log_id	appointment_id, changed_by, old_status_id, new_status_id	timestamp default, nullable old/new status

2.1 Relationship and constraint strategy

- Primary keys uniquely identify every row and keep entity boundaries clear.
- Foreign keys enforce referential integrity and prevent broken references.
- UNIQUE constraints protect natural identifiers such as user email and provider license number.
- CHECK constraints enforce role values, status logic, valid day names, and valid time ranges.
- ON DELETE CASCADE is used only where dependent records should be removed together; ON DELETE RESTRICT is used when deletion would leave the operational schedule in an invalid state.

2.2 Full CREATE TABLE statements

The following SQL statements implement the final relational schema used for Sprint 2.

USER

```sql
CREATE TABLE "USER" (
    user_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    role VARCHAR(20) NOT NULL
        CHECK (role IN ('patient', 'provider', 'admin')),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

PATIENT

```
CREATE TABLE PATIENT (
    patient_id INT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(10)
        CHECK (gender IN ('male', 'female', 'other')),
    emergency_contact_name VARCHAR(100),
    emergency_contact_phone VARCHAR(20),
    insurance_provider VARCHAR(100),
    insurance_policy_number VARCHAR(50),
    FOREIGN KEY (user_id) REFERENCES "USER"(user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

DEPARTMENT

```
CREATE TABLE DEPARTMENT (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE,
    location VARCHAR(200),
    phone VARCHAR(20)
);
```

PROVIDER

```
CREATE TABLE PROVIDER (
    provider_id INT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    department_id INT NOT NULL,
    specialty VARCHAR(100) NOT NULL,
    license_number VARCHAR(50) NOT NULL UNIQUE,
    FOREIGN KEY (user_id) REFERENCES "USER"(user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (department_id) REFERENCES DEPARTMENT(department_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

APPOINTMENT_STATUS

```
CREATE TABLE APPOINTMENT_STATUS (
    status_id INT PRIMARY KEY,
    status_name VARCHAR(30) NOT NULL UNIQUE
);
```

APPOINTMENT

```
CREATE TABLE APPOINTMENT (
    appointment_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    provider_id INT NOT NULL,
    department_id INT NOT NULL,
    status_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    reason VARCHAR(255),
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (end_time > start_time),
    FOREIGN KEY (patient_id) REFERENCES PATIENT(patient_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
```

```
    FOREIGN KEY (provider_id) REFERENCES PROVIDER(provider_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (department_id) REFERENCES DEPARTMENT(department_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (status_id) REFERENCES APPOINTMENT_STATUS(status_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

PROVIDER_AVAILABILITY

```
CREATE TABLE PROVIDER_AVAILABILITY (
    availability_id INT PRIMARY KEY,
    provider_id INT NOT NULL,
    day_of_week VARCHAR(10) NOT NULL
        CHECK (day_of_week IN (
            'Monday', 'Tuesday', 'Wednesday', 'Thursday',
            'Friday', 'Saturday', 'Sunday'
        )),
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    CHECK (end_time > start_time),
    FOREIGN KEY (provider_id) REFERENCES PROVIDER(provider_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

APPOINTMENT_AUDIT_LOG

```
CREATE TABLE APPOINTMENT_AUDIT_LOG (
    log_id INT PRIMARY KEY,
    appointment_id INT NOT NULL,
    changed_by INT NOT NULL,
    old_status_id INT,
    new_status_id INT,
    change_timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    change_reason VARCHAR(255),
    FOREIGN KEY (appointment_id) REFERENCES APPOINTMENT(appointment_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (changed_by) REFERENCES "USER"(user_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (old_status_id) REFERENCES APPOINTMENT_STATUS(status_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    FOREIGN KEY (new_status_id) REFERENCES APPOINTMENT_STATUS(status_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

3. Business Rules Enforcement

The database enforces the following business rules through constraints, relational design choices, and supported query logic.

Business Rule	How It Is Enforced
Every appointment must be linked to a valid patient, provider, department, and status	NOT NULL foreign keys in APPOINTMENT plus FK constraints that reject invalid references.
Appointment end time must be after start time	CHECK (end_time > start_time) in APPOINTMENT.

Business Rule	How It Is Enforced
Each user must have a unique email	UNIQUE constraint on USER.email.
A patient can map to only one user account	UNIQUE constraint on PATIENT.user_id.
A provider can map to only one user account	UNIQUE constraint on PROVIDER.user_id.
Each provider must have a unique license number	UNIQUE constraint on PROVIDER.license_number.
A provider cannot be deleted while referenced by appointments	ON DELETE RESTRICT on APPOINTMENT.provider_id.
A department cannot be deleted while referenced by providers or appointments	ON DELETE RESTRICT on PROVIDER.department_id and APPOINTMENT.department_id.
Provider availability must use valid day names and valid time ranges	CHECK on day_of_week and CHECK (end_time > start_time) in PROVIDER_AVAILABILITY.
Appointment statuses must come from a predefined lookup table	Foreign key from APPOINTMENT.status_id to APPOINTMENT_STATUS.status_id.

4. Edge Cases and Failure Handling

The following scenarios represent realistic failures or conflict situations that the system must handle correctly.

Scenario	What Can Go Wrong?	How the System Handles It
Two appointments are booked for the same provider at overlapping times	The provider becomes double-booked.	The application checks for overlap inside a transaction before inserting; conflicting inserts are rejected.
A provider with future appointments is deleted	Appointments would lose their provider reference.	ON DELETE RESTRICT on APPOINTMENT.provider_id blocks the delete.
A department with assigned providers is deleted	Providers would reference a missing department.	ON DELETE RESTRICT on PROVIDER.department_id prevents deletion.
An appointment uses a nonexistent patient or provider ID	The appointment references an invalid record.	Foreign key validation causes the insert to fail.
An appointment is booked outside the provider's working hours	A patient is scheduled when the provider is unavailable.	The application checks PROVIDER_AVAILABILITY before creating the appointment.
A status value outside the lookup table is assigned	Appointments receive meaningless or inconsistent statuses.	Foreign key validation rejects any status not present in APPOINTMENT_STATUS.
A user with a patient profile and appointments is deleted	Dependent records could become orphaned.	ON DELETE CASCADE removes the patient, appointments, and audit log entries in a controlled order.
Two users submit the same provider booking at the same time	Both inserts could pass the overlap check and commit.	The overlap check and insert run inside a serializable transaction so only one succeeds.

5. Normalization and Functional Dependencies

Normalization was used to transform the scheduling domain into a set of lossless, non-redundant relations. The final design removes repeated textual values, separates independent fact sets, and preserves integrity through primary and foreign keys.

5.1 Design evolution from raw scheduling data

- A single unnormalized scheduling record would mix user data, patient details, provider details, department details, status text, appointment facts, availability windows, and audit history in the same structure.
- To reach 1NF, repeating and multivalued facts were separated so that one row stores one fact about one entity or event.
- To reach 2NF and 3NF, user account data, provider data, department data, and appointment status values were decomposed into dedicated relations so that non-key attributes depend only on the key.
- To support 4NF and 5NF quality, provider availability and appointment audit history were separated from APPOINTMENT because they represent independent many-valued relationships that would otherwise create duplication and update anomalies.

5.2 Functional dependency summary

Table	Primary Key	Functional Dependencies
USER	user_id	user_id → first_name, last_name, email, password_hash, phone, role, created_at
PATIENT	patient_id	patient_id → user_id, date_of_birth, gender, emergency_contact_name, emergency_contact_phone, insurance_provider, insurance_policy_number
DEPARTMENT	department_id	department_id → department_name, location, phone
PROVIDER	provider_id	provider_id → user_id, department_id, specialty, license_number
APPOINTMENT_STATUS	status_id	status_id → status_name
APPOINTMENT	appointment_id	appointment_id → patient_id, provider_id, department_id, status_id, appointment_date, start_time, end_time, reason, notes, created_at
PROVIDER_AVAILABILITY	availability_id	availability_id → provider_id, day_of_week, start_time, end_time
APPOINTMENT_AUDIT_LOG	log_id	log_id → appointment_id, changed_by, old_status_id, new_status_id, change_timestamp, change_reason

5.3 Text-based functional dependency diagrams

```
USER
  user_id ──▶ first_name, last_name, email, password_hash, phone, role, created_at
```

```
PATIENT
  patient_id ──▶ user_id, date_of_birth, gender, emergency_contact_name,
                 emergency_contact_phone, insurance_provider, insurance_policy_number

DEPARTMENT
  department_id ──▶ department_name, location, phone

PROVIDER
  provider_id ──▶ user_id, department_id, specialty, license_number

APPOINTMENT_STATUS
  status_id ──▶ status_name

APPOINTMENT
  appointment_id ──▶ patient_id, provider_id, department_id, status_id,
                     appointment_date, start_time, end_time, reason, notes, created_at

PROVIDER_AVAILABILITY
  availability_id ──▶ provider_id, day_of_week, start_time, end_time

APPOINTMENT_AUDIT_LOG
  log_id ──▶ appointment_id, changed_by, old_status_id, new_status_id,
             change_timestamp, change_reason
```

5.4 Normal form analysis

Normal Form	Relations Reviewed	Result
1NF	All relations	All attributes are atomic. Repeating groups such as status labels, department facts, provider availability windows, and audit history are stored in separate tables rather than multi-valued columns.
2NF	All relations	Every table uses a single-attribute primary key, so no non-key attribute can depend on only part of a composite key. This eliminates partial dependency issues by design.
3NF	USER / PATIENT / PROVIDER / DEPARTMENT / APPOINTMENT_STATUS / APPOINTMENT	Transitive dependencies were removed by separating shared user data, department data, and status labels into their own relations. APPOINTMENT stores only foreign keys to those dimensions.
BCNF	Final schema	Every determinant in the final schema is a candidate key for its relation. Business identifiers such as USER.email and PROVIDER.license_number are protected with UNIQUE constraints.
4NF	PROVIDER_AVAILABILITY / APPOINTMENT_AUDIT_LOG	Independent multi-valued facts were decomposed. A provider can have many availability windows, and an appointment can have many audit events, but those sets are stored outside the main transaction row.
5NF	Final schema	No remaining join dependency justifies further decomposition. Reconstructing the full business view

Normal Form	Relations Reviewed	Result
		requires natural joins across losslessly decomposed tables.

5.5 Key normalization outcomes

- Status text was separated into APPOINTMENT_STATUS so appointment rows store stable foreign keys instead of repeating labels.
- Department facts were separated into DEPARTMENT so provider and appointment rows do not duplicate location and department contact information.
- Shared account data was centralized in USER, while PATIENT and PROVIDER store only role-specific attributes.
- Provider availability was separated from APPOINTMENT because availability windows are independent from booked appointment transactions.
- Audit history was separated into APPOINTMENT_AUDIT_LOG to preserve a time-based change trail without overwriting the current appointment state.

6. Data Operations (CRUD with Real Scenarios)

The following twelve SQL statements represent realistic insert, update, delete, and retrieval actions supported by the system.

6.1 INSERT: A new user registers an account.

```sql
INSERT INTO "USER" (
    user_id, first_name, last_name, email, password_hash, phone, role, created_at
) VALUES (
    1, 'John', 'Doe', 'john.doe@email.com', 'hashed_pw_123',
    '509-555-0101', 'patient', CURRENT_TIMESTAMP
);
```

Expected behavior: A new USER row is created. If the email already exists, the insert fails because of the UNIQUE constraint.

6.2 INSERT: The new user receives a patient profile.

```sql
INSERT INTO PATIENT (
    patient_id, user_id, date_of_birth, gender, emergency_contact_name,
    emergency_contact_phone, insurance_provider, insurance_policy_number
) VALUES (
    1, 1, '1990-05-15', 'male', 'Jane Doe', '509-555-0102',
    'Blue Cross', 'BC-998877'
);
```

Expected behavior: A PATIENT record is created and linked to user_id 1. UNIQUE(user_id) ensures one patient profile per user.

6.3 INSERT: A patient books a new appointment.

```sql
INSERT INTO APPOINTMENT (
    appointment_id, patient_id, provider_id, department_id, status_id,
    appointment_date, start_time, end_time, reason, notes, created_at
) VALUES (
    1, 1, 1, 1, 1, '2026-04-15', '09:00', '09:30',
    'Annual checkup', NULL, CURRENT_TIMESTAMP
);
```

Expected behavior: The appointment is created only if every referenced patient, provider, department, and status exists.

6.4 INSERT: An audit log entry is created when the appointment is booked.

```
INSERT INTO APPOINTMENT_AUDIT_LOG (
    log_id, appointment_id, changed_by, old_status_id, new_status_id,
    change_timestamp, change_reason
) VALUES (
    1, 1, 1, NULL, 1, CURRENT_TIMESTAMP, 'Appointment created'
);
```

Expected behavior: A status-history record is inserted with old_status_id = NULL because the appointment is being created.

6.5 UPDATE: A patient reschedules an appointment.

```
UPDATE APPOINTMENT
SET appointment_date = '2026-04-20',
    start_time = '10:00',
    end_time = '10:30'
WHERE appointment_id = 1;
```

Expected behavior: The date and times are updated. The CHECK constraint still enforces end_time > start_time.

6.6 UPDATE: A provider marks an appointment as completed.

```
UPDATE APPOINTMENT
SET status_id = 2
WHERE appointment_id = 1;
```

Expected behavior: The status changes from Scheduled to Completed only if status_id 2 exists in APPOINTMENT_STATUS.

6.7 UPDATE: A provider changes Monday availability.

```
UPDATE PROVIDER_AVAILABILITY
SET start_time = '08:00',
    end_time = '16:00'
WHERE provider_id = 1
  AND day_of_week = 'Monday';
```

Expected behavior: The provider availability window is updated while preserving valid time ordering.

6.8 DELETE: A patient deletes their account entirely.

```
DELETE FROM "USER"
WHERE user_id = 1;
```

Expected behavior: Cascade rules remove the patient record, appointments, and related audit log entries in a consistent order.

6.9 DELETE: An administrator removes an old availability slot.

```
DELETE FROM PROVIDER_AVAILABILITY
WHERE availability_id = 3;
```

Expected behavior: The selected availability record is removed without affecting unrelated tables.

6.10 DELETE: An administrator tries to delete a department that still has providers.

```
DELETE FROM DEPARTMENT
WHERE department_id = 1;
```

Expected behavior: The delete is rejected because PROVIDER.department_id uses ON DELETE RESTRICT.

6.11 SELECT: A patient views upcoming scheduled appointments.

```
SELECT appointment_id, appointment_date, start_time, end_time, reason
FROM APPOINTMENT
WHERE patient_id = 1
  AND appointment_date >= CURRENT_DATE
  AND status_id = 1
ORDER BY appointment_date, start_time;
```

Expected behavior: Returns all future scheduled appointments for patient 1, ordered by date and time.

6.12 SELECT: An administrator finds patients who have never booked an appointment.

```
SELECT p.patient_id, u.first_name, u.last_name
FROM PATIENT p
JOIN "USER" u ON p.user_id = u.user_id
LEFT JOIN APPOINTMENT a ON p.patient_id = a.patient_id
WHERE a.appointment_id IS NULL;
```

Expected behavior: Returns patients with zero appointments by keeping unmatched rows with a LEFT JOIN.

7. Advanced Retrieval Queries

These ten queries demonstrate real application usage across multiple related tables. Every query uses JOINs, and aggregate queries use GROUP BY where appropriate.

7.1 Query 1: View all upcoming appointments with patient and provider names.

```
SELECT a.appointment_id, a.appointment_date, a.start_time, a.end_time,
       pu.first_name AS patient_first, pu.last_name AS patient_last,
       pru.first_name AS provider_first, pru.last_name AS provider_last
FROM APPOINTMENT a
JOIN PATIENT pt ON a.patient_id = pt.patient_id
JOIN "USER" pu ON pt.user_id = pu.user_id
JOIN PROVIDER pr ON a.provider_id = pr.provider_id
JOIN "USER" pru ON pr.user_id = pru.user_id
WHERE a.appointment_date >= CURRENT_DATE
ORDER BY a.appointment_date, a.start_time;
```

Why JOIN is required: Patient and provider names are stored in USER, so APPOINTMENT must join through PATIENT and PROVIDER to resolve names.

Why GROUP BY is used: Not needed because the query lists detail rows rather than grouped summaries.

Expected output: A readable schedule of future appointments with patient and provider names.

7.2 Query 2: Find all providers in Cardiology along with weekly availability.

```
SELECT pru.first_name, pru.last_name, pr.specialty,
       pa.day_of_week, pa.start_time, pa.end_time
FROM PROVIDER pr
JOIN "USER" pru ON pr.user_id = pru.user_id
```

```
JOIN DEPARTMENT d ON pr.department_id = d.department_id
LEFT JOIN PROVIDER_AVAILABILITY pa ON pr.provider_id = pa.provider_id
WHERE d.department_name = 'Cardiology'
ORDER BY pru.last_name, pa.day_of_week;
```

Why JOIN is required: Names come from USER, department names from DEPARTMENT, and work windows from PROVIDER_AVAILABILITY.

Why GROUP BY is used: Not used because each provider-availability row is shown individually.

Expected output: Cardiology providers and their work windows; providers without availability still appear.

7.3 Query 3: Get one patient's appointment history with status names and provider names.

```
SELECT a.appointment_id, a.appointment_date, a.start_time, a.reason,
       s.status_name,
       pru.first_name AS provider_first, pru.last_name AS provider_last
FROM APPOINTMENT a
JOIN APPOINTMENT_STATUS s ON a.status_id = s.status_id
JOIN PROVIDER pr ON a.provider_id = pr.provider_id
JOIN "USER" pru ON pr.user_id = pru.user_id
WHERE a.patient_id = 1
ORDER BY a.appointment_date DESC;
```

Why JOIN is required: Status labels live in APPOINTMENT_STATUS and provider names live in USER through PROVIDER.

Why GROUP BY is used: Not needed because the result is a detailed history, not an aggregate.

Expected output: The selected patient's appointments with human-readable status labels and provider names.

7.4 Query 4: Count how many appointments each department has.

```
SELECT d.department_name,
       COUNT(a.appointment_id) AS total_appointments
FROM DEPARTMENT d
LEFT JOIN APPOINTMENT a ON d.department_id = a.department_id
GROUP BY d.department_name
ORDER BY total_appointments DESC;
```

Why JOIN is required: Department names come from DEPARTMENT while appointment facts come from APPOINTMENT.

Why GROUP BY is used: GROUP BY rolls appointment rows up to one summary row per department.

Expected output: Each department and its appointment count, including departments with zero appointments.

7.5 Query 5: Find all cancelled appointments and identify who cancelled them.

```
SELECT a.appointment_id, a.appointment_date,
       pu.first_name AS patient_first, pu.last_name AS patient_last,
       cu.first_name AS cancelled_by_first, cu.last_name AS cancelled_by_last,
       al.change_timestamp, al.change_reason
FROM APPOINTMENT a
JOIN APPOINTMENT_AUDIT_LOG al ON a.appointment_id = al.appointment_id
JOIN APPOINTMENT_STATUS ns ON al.new_status_id = ns.status_id
JOIN PATIENT pt ON a.patient_id = pt.patient_id
JOIN "USER" pu ON pt.user_id = pu.user_id
JOIN "USER" cu ON al.changed_by = cu.user_id
WHERE ns.status_name = 'Cancelled';
```

Why JOIN is required: The audit log records the user who made the change, and status labels require the status lookup table.

Why GROUP BY is used: Not required because the query lists each cancellation event.

Expected output: Cancelled appointments with patient name, cancelling user, timestamp, and reason.

7.6 Query 6: Detect double-bookings by finding overlapping appointments for the same provider.

```
SELECT a1.appointment_id AS appt1, a2.appointment_id AS appt2,
       a1.appointment_date, a1.start_time AS start1, a1.end_time AS end1,
       a2.start_time AS start2, a2.end_time AS end2,
       pru.first_name, pru.last_name
FROM APPOINTMENT a1
JOIN APPOINTMENT a2
  ON a1.provider_id = a2.provider_id
 AND a1.appointment_date = a2.appointment_date
 AND a1.appointment_id < a2.appointment_id
 AND a1.start_time < a2.end_time
 AND a2.start_time < a1.end_time
JOIN PROVIDER pr ON a1.provider_id = pr.provider_id
JOIN "USER" pru ON pr.user_id = pru.user_id;
```

Why JOIN is required: A self-join compares appointment pairs for the same provider on the same date, then joins resolve the provider's name.

Why GROUP BY is used: Not used because the goal is to show conflicting detail rows, not grouped summaries.

Expected output: Any overlapping appointment pairs for a provider; ideally no rows are returned.

7.7 Query 7: Find providers who currently have zero appointments scheduled.

```
SELECT pru.first_name, pru.last_name, pr.specialty, d.department_name
FROM PROVIDER pr
JOIN "USER" pru ON pr.user_id = pru.user_id
JOIN DEPARTMENT d ON pr.department_id = d.department_id
LEFT JOIN APPOINTMENT a ON pr.provider_id = a.provider_id
WHERE a.appointment_id IS NULL;
```

Why JOIN is required: The provider name and department live outside APPOINTMENT, so the query joins USER and DEPARTMENT and uses a LEFT JOIN to detect missing appointments.

Why GROUP BY is used: Not used because the query searches for unmatched rows rather than aggregates.

Expected output: Providers without appointments, including specialty and department.

7.8 Query 8: Show the full audit trail for a specific appointment.

```
SELECT al.log_id, al.change_timestamp,
       os.status_name AS old_status,
       ns.status_name AS new_status,
       u.first_name AS changed_by_first,
       u.last_name AS changed_by_last,
       al.change_reason
FROM APPOINTMENT_AUDIT_LOG al
JOIN "USER" u ON al.changed_by = u.user_id
LEFT JOIN APPOINTMENT_STATUS os ON al.old_status_id = os.status_id
LEFT JOIN APPOINTMENT_STATUS ns ON al.new_status_id = ns.status_id
WHERE al.appointment_id = 1
ORDER BY al.change_timestamp;
```

Why JOIN is required: Old and new status labels live in APPOINTMENT_STATUS, and the user who made the change lives in USER.

Why GROUP BY is used: Not used because each audit event should remain visible.

Expected output: A chronological status-change history for one appointment.

7.9 Query 9: Find patients with more than three appointments in the last 30 days.

```
SELECT pu.first_name, pu.last_name,
       COUNT(a.appointment_id) AS appt_count
FROM APPOINTMENT a
JOIN PATIENT pt ON a.patient_id = pt.patient_id
JOIN "USER" pu ON pt.user_id = pu.user_id
WHERE a.appointment_date >= CURRENT_DATE - INTERVAL '30' DAY
GROUP BY pu.first_name, pu.last_name
HAVING COUNT(a.appointment_id) > 3;
```

Why JOIN is required: The appointment count must be tied back to the patient name through PATIENT and USER.

Why GROUP BY is used: GROUP BY and HAVING are required to count appointments per patient and filter to high-frequency patients.

Expected output: Patients with a high appointment volume in the last 30 days.

7.10 Query 10: List today's full schedule with department, provider, patient, and status.

```
SELECT a.appointment_id, a.start_time, a.end_time, a.reason,
       pu.first_name AS patient_first, pu.last_name AS patient_last,
       pru.first_name AS provider_first, pru.last_name AS provider_last,
       d.department_name, s.status_name
FROM APPOINTMENT a
JOIN PATIENT pt ON a.patient_id = pt.patient_id
JOIN "USER" pu ON pt.user_id = pu.user_id
JOIN PROVIDER pr ON a.provider_id = pr.provider_id
JOIN "USER" pru ON pr.user_id = pru.user_id
JOIN DEPARTMENT d ON a.department_id = d.department_id
JOIN APPOINTMENT_STATUS s ON a.status_id = s.status_id
WHERE a.appointment_date = CURRENT_DATE
ORDER BY a.start_time;
```

Why JOIN is required: APPOINTMENT stores IDs, so joins are required to resolve the patient name, provider name, department, and status label.

Why GROUP BY is used: Not used because the query is a readable operational schedule, not an aggregate.

Expected output: Every appointment scheduled for today with all important labels resolved.

8. Validation with Sample Data

Meaningful seed data was inserted to validate both normal workflows and failure cases. The sample data includes departments, appointment statuses, users, patients, providers, provider availability windows, appointments, and audit log entries.

8.1 Seed data

```
INSERT INTO APPOINTMENT_STATUS VALUES (1, 'Scheduled');
INSERT INTO APPOINTMENT_STATUS VALUES (2, 'Completed');
INSERT INTO APPOINTMENT_STATUS VALUES (3, 'Cancelled');
INSERT INTO APPOINTMENT_STATUS VALUES (4, 'No-Show');
```

```
INSERT INTO DEPARTMENT VALUES (1, 'Cardiology', 'Building A, Floor 2', '509-555-1001');
INSERT INTO DEPARTMENT VALUES (2, 'Pediatrics', 'Building B, Floor 1', '509-555-1002');
INSERT INTO DEPARTMENT VALUES (3, 'Dermatology', 'Building A, Floor 3', '509-555-1003');

INSERT INTO "USER" VALUES (1, 'John', 'Doe', 'john.doe@email.com', 'hash1', '509-555-0101',
'patient', '2026-01-10 08:00:00');
INSERT INTO "USER" VALUES (2, 'Jane', 'Smith', 'jane.smith@email.com', 'hash2', '509-555-0102',
'patient', '2026-01-11 09:00:00');
INSERT INTO "USER" VALUES (3, 'Dr. Alice', 'Brown', 'alice.brown@email.com', 'hash3',
'509-555-0201', 'provider', '2026-01-05 08:00:00');
INSERT INTO "USER" VALUES (4, 'Dr. Bob', 'Lee', 'bob.lee@email.com', 'hash4', '509-555-0202',
'provider', '2026-01-06 08:00:00');
INSERT INTO "USER" VALUES (5, 'Admin', 'User', 'admin@hospital.com', 'hash5', '509-555-0300',
'admin', '2026-01-01 08:00:00');
INSERT INTO "USER" VALUES (6, 'Emily', 'Clark', 'emily.clark@email.com', 'hash6', '509-555-0103',
'patient', '2026-02-01 10:00:00');

INSERT INTO PATIENT VALUES (1, 1, '1990-05-15', 'male', 'Jane Doe', '509-555-0102', 'Blue Cross',
'BC-998877');
INSERT INTO PATIENT VALUES (2, 2, '1985-11-20', 'female', 'Bob Smith', '509-555-0110', 'Aetna',
'AE-112233');
INSERT INTO PATIENT VALUES (3, 6, '2000-03-08', 'female', 'Tom Clark', '509-555-0120', 'United
Health', 'UH-445566');

INSERT INTO PROVIDER VALUES (1, 3, 1, 'Cardiologist', 'LIC-0001');
INSERT INTO PROVIDER VALUES (2, 4, 2, 'Pediatrician', 'LIC-0002');

INSERT INTO PROVIDER_AVAILABILITY VALUES (1, 1, 'Monday', '08:00', '16:00');
INSERT INTO PROVIDER_AVAILABILITY VALUES (2, 1, 'Wednesday', '08:00', '16:00');
INSERT INTO PROVIDER_AVAILABILITY VALUES (3, 1, 'Friday', '08:00', '12:00');
INSERT INTO PROVIDER_AVAILABILITY VALUES (4, 2, 'Tuesday', '09:00', '17:00');
INSERT INTO PROVIDER_AVAILABILITY VALUES (5, 2, 'Thursday', '09:00', '17:00');

INSERT INTO APPOINTMENT VALUES (1, 1, 1, 1, 1, '2026-04-15', '09:00', '09:30', 'Annual checkup',
NULL, '2026-04-01 10:00:00');
INSERT INTO APPOINTMENT VALUES (2, 2, 1, 1, 2, '2026-04-10', '10:00', '10:45', 'Follow-up ECG',
'Results normal', '2026-03-28 14:00:00');
INSERT INTO APPOINTMENT VALUES (3, 1, 2, 2, 3, '2026-04-12', '11:00', '11:30', 'Child wellness
visit', NULL, '2026-04-01 11:00:00');
INSERT INTO APPOINTMENT VALUES (4, 3, 1, 1, 1, '2026-04-15', '10:00', '10:30', 'Heart palpitations',
NULL, '2026-04-02 09:00:00');
INSERT INTO APPOINTMENT VALUES (5, 2, 2, 2, 1, '2026-04-17', '09:00', '09:30', 'Vaccination', NULL,
'2026-04-03 08:00:00');

INSERT INTO APPOINTMENT_AUDIT_LOG VALUES (1, 1, 1, NULL, 1, '2026-04-01 10:00:00', 'Appointment
created');
INSERT INTO APPOINTMENT_AUDIT_LOG VALUES (2, 2, 2, NULL, 1, '2026-03-28 14:00:00', 'Appointment
created');
INSERT INTO APPOINTMENT_AUDIT_LOG VALUES (3, 2, 3, 1, 2, '2026-04-10 10:50:00', 'Visit completed');
INSERT INTO APPOINTMENT_AUDIT_LOG VALUES (4, 3, 1, 1, 3, '2026-04-11 08:00:00', 'Patient requested
cancellation');
INSERT INTO APPOINTMENT_AUDIT_LOG VALUES (5, 4, 6, NULL, 1, '2026-04-02 09:00:00', 'Appointment
created');
```

8.2 Sample output: upcoming appointments

Appt ID	Date	Start	End	Patient	Provider
1	2026-04-15	09:00	09:30	John Doe	Dr. Alice Brown
4	2026-04-15	10:00	10:30	Emily Clark	Dr. Alice Brown
5	2026-04-17	09:00	09:30	Jane Smith	Dr. Bob Lee

8.3 Sample output: appointments per department

Department	Total Appointments
Cardiology	3
Pediatrics	2
Dermatology	0

8.4 Sample output: audit trail for appointment 2

Log ID	Timestamp	Old	New	Changed By	Reason
2	2026-03-28 14:00	NULL	Scheduled	Jane Smith	Appointment created
3	2026-04-10 10:50	Scheduled	Completed	Dr. Alice Brown	Visit completed

8.5 Constraint tests

Test	SQL	Expected Result
Insert appointment where end_time < start_time	INSERT INTO APPOINTMENT VALUES (99, 1, 1, 1, 1, '2026-05-01', '10:00', '09:00', 'Bad time', NULL, CURRENT_TIMESTAMP);	Fails because CHECK (end_time > start_time) is violated.
Delete a department that still has providers	DELETE FROM DEPARTMENT WHERE department_id = 1;	Fails because ON DELETE RESTRICT blocks the deletion.

8.6 System integrity reflection

Most critical business rule: Every appointment must reference a valid patient, provider, department, and status. Without those links, the scheduling record loses operational meaning.

Hardest edge case: Preventing overlapping appointments for the same provider is the hardest case because a row-level CHECK constraint cannot compare against other rows. The conflict check must be performed inside application logic and a transaction.

Concurrent actions: If two users attempt to book the same provider and slot simultaneously, the overlap check and insert should be wrapped inside a serializable transaction so that only one commit succeeds.

How invalid states are prevented: NOT NULL, UNIQUE, CHECK, and foreign key constraints reject incomplete, duplicate, illogical, or disconnected data. RESTRICT rules block unsafe deletions.

9. UI-Based System Validation

The Sprint 2 assignment also requires a basic UI capable of exercising the database. This section documents the required behaviors the UI should demonstrate during the live demo or in screenshots.

9.1 CRUD support in the UI

Capability	Example UI Action	Validation Behavior
Create / Insert	Register user, create patient or provider profile, schedule appointment	Client-side required-field checks plus database constraints

Capability	Example UI Action	Validation Behavior
Read / Retrieve	View upcoming appointments, patient history, provider availability, daily schedule	Search and reporting pages backed by SELECT and JOIN queries
Update / Modify	Reschedule appointment, update status, edit availability	Invalid time windows fail because of CHECK constraints and validation
Delete / Remove	Delete availability slot or user account, attempt to delete restricted department/provider	Dependency-aware deletes show friendly errors when RESTRICT rules block the action

9.2 Business-rule enforcement shown through the UI

- Required fields such as patient, provider, appointment date, start time, end time, and status should be validated before submission.
- The UI should display clear messages when the database rejects invalid references, duplicate values, or invalid time ranges.
- Delete actions should explain why the operation is blocked when dependent records still exist.

9.3 Edge cases to demonstrate through the UI

Edge Case	User Action	Expected System Response
Duplicate email during account creation	User submits an email already in USER.	Display a validation message explaining that the email address is already in use.
Invalid appointment time	User enters an end time earlier than the start time.	Reject submission and highlight the time fields.
Delete dependent department	Admin attempts to delete a department with assigned providers or appointments.	Show a dependency error and keep data unchanged.
Conflicting provider booking	Two users try to reserve the same provider slot.	Application overlap check rejects one request and prompts the user to choose a different time.

9.4 Screenshots

- Will be added in the submission.

10. Conclusion

The Healthcare Appointment Scheduling System now includes a normalized relational schema, validated SQL implementation, business-rule enforcement, realistic edge-case handling, CRUD workflows, advanced JOIN-based reporting, and a UI validation plan. Together, these artifacts demonstrate that the project satisfies the Sprint 2 objectives and is ready for refinement in Sprint 3.

Appendix A. Quick Reference to Required Deliverables

Assignment Component	Included in This Report
CREATE TABLE statements	Yes — Section 2.2
Primary keys, foreign keys, and constraints	Yes — Sections 2 and 3

Assignment Component	Included in This Report
8–10 business rules	Yes — Section 3
6–8 edge cases	Yes — Section 4
Functional dependencies and normalization	Yes — Section 5
At least 12 CRUD SQL queries	Yes — Section 6
10 advanced JOIN-based queries	Yes — Section 7
Sample data and validation outputs	Yes — Section 8
UI support for CRUD, edge cases, and validation	Yes — Section 9